

test rules 파이썬 입문 가이드

목표 테스트 코드를 읽으며 함수 호출과 결과 검증을 이해하기

먼저 이해할 결론. test_rules.py는 주문 수량 계산 규칙이 예상대로 동작하는지 확인하는 자동 테스트 파일입니다. 수량 규칙을 외우기보다, 각 테스트가 준비 → 실행 → 확인 순서로 구성된다는 점을 먼저 읽어 보세요.

1 파일의 큰 구조

부분	초보자 관점의 의미
import	다른 파일의 도구를 가져옵니다. Decimal, unittest, 주문 관련 클래스가 여기에 해당합니다.
class QuantityRuleTests	여러 테스트를 한 묶음으로 관리하는 상자입니다.
def decide	반복되는 기본 입력값을 준비하는 보조 함수입니다.
def test_...	한 상황을 실행하고, 결과가 맞는지 확인하는 테스트 함수입니다.
unittest.main	이 파일을 직접 실행했을 때 테스트를 실행합니다.

2 import 와 별칭

```
from decimal import Decimal as D
import unittest
from mkr_sync.rules import build_decisions, decide_quantity
```

from A import B는 A 안의 B만 가져오겠다는 뜻입니다. as D는 Decimal이라는 긴 이름을 이 파일에서는 D로 줄여 쓰겠다는 별칭입니다. 따라서 D("100")은 정확한 10진수 100을 만드는 표현입니다.

3 클래스와 테스트 함수

```
class QuantityRuleTests(unittest.TestCase):
    def test_rule2_already_net(self):
        result = self.decide(...)
        self.assertEqual(result.rule_id, "R2_ALREADY_NET")
```

class는 관련 기능을 묶는 상자입니다. 괄호 안의 unittest.TestCase는 이 클래스가 테스트 기능을 물려받는다는 뜻입니다. def로 시작하는 test_ 함수는 하나의 검증 시나리오이고, unittest는 test_로 시작하는 함수를 찾아 실행합니다. self는 현재 테스트 객체 자신을 가리키므로 self.decide(...)는 같은 클래스의 보조 함수를 호출한다는 뜻입니다.

4 딕셔너리와 키워드 인수

```
values = {"mkr": "MKR56/26", "raw_order_qty": D("100")}
values.update(overrides)
return decide_quantity(**values)
```

딕셔너리는 이름표와 값을 연결한 자료형입니다. values에는 기본 입력값을 모아 둡니다. values.update(overrides)는 뒤에서 받은 변경값으로 기본값을 덮어씁니다. **values는 딕셔너리를 함수의 키워드 인수로 풀어 주는 문법입니다.

5 한 테스트를 줄 단위로 읽기

```
def test_rule2_already_net(self):
    result = self.decide(
        backorder_order_qty=D("40"),
        backorder_shipped_qty=D("20"),
    )
    self.assertEqual(result.rule_id, "R2_ALREADY_NET")
    self.assertEqual(result.order_qty, D("100"))
```

첫 줄은 테스트 이름입니다. 다음 부분은 기본 입력값에 백오더 수량 두 개만 바꿔 decide 함수를 실행합니다. 반환된 결과 객체를 result 변수에 담고, 마지막 두 줄에서 실제 결과와 기대값이 같은지 확인합니다.

문법	읽는 방법
=	오른쪽 값을 왼쪽 변수에 저장합니다. result = ...
.	객체 안의 정보에 접근합니다. result.rule_id
assertEqual	실제값과 기대값이 다르면 테스트를 실패시킵니다.
D("100")	정확한 10진수 100을 만듭니다. 수량 계산에 적합합니다.

6 예외가 나와 성공하는 테스트

```
with self.assertRaises(QuantityRuleError):
    self.decide(backorder_order_qty=D("101"))
```

잘못된 수량 조합은 QuantityRuleError로 중단되어야 합니다. with self.assertRaises(...)는 블록 안에서 지정한 오류가 발생하면 테스트를 성공으로 처리합니다. 이는 프로그램이 망가졌다는 뜻이 아니라, 잘못된 데이터를 조용히 통과시키지 않는다는 확인입니다.

7 None 과 True False

```
sales_note_order_qty=None
self.assertIsNone(result.shipped_qty)
self.assertFalse(result.shipped_fallback)
```

None은 값이 0이라는 뜻이 아니라 아직 알 수 없거나 값이 없다는 뜻입니다. Sales Note가 없을 때 shipped_qty와 shortage_qty를 None으로 두면, 출하량이 0이라고 오해하지 않습니다. True와 False는 조건의 참과 거짓을 표현하는 bool 값입니다.

8 추천 읽기 순서

처음에는 decide 함수와 test_rule2_already_net만 함께 보세요. 그다음

test_missing_sales_note_keeps_shipment_and_shortage_unknown을 읽으면 None을 실제 코드에서 왜 쓰는지 확인할 수 있습니다. 각 테스트를 읽을 때는 입력값, 실행 함수, 기대 결과 세 부분에 표시하며 따라가면 됩니다.

작은 연습 test_rule5_raw_includes_I에서 decide에 전달한 값을 찾아 적어 보고, assertEquals 안의 실제값과 기대값이 무엇인지 각각 말로 설명해 보세요.